DSC 291: Learning Theory Assignment 1

Tongle Shen

April 10, 2026

1 Part A: Repository Setup

Repository link:
https://github.com/Ghostlikei/AI4LT

2 Part B: Theory Problem

2.1 Problem 1

2.1.1 Choice of Grid

Let

$$m = \left\lceil \frac{1}{\Delta} \right\rceil \quad \text{and} \quad G = \left\{ \frac{k}{m} : k = 0, 1, \ldots, m \right\}.$$

Then $G \subseteq [0, 1]$ is a finite grid with

$$|G| = m + 1 = \left\lceil \frac{1}{\Delta} \right\rceil + 1.$$

2.1.2 Existence of a Consistent Grid Threshold

We claim that for every Δ-separated threshold-realizable sequence, there exists some $\widetilde{\theta} \in G$ such that

$$h_{\widetilde{\theta}}(x_t) = y_t \quad \text{for all } t = 1, \ldots, T.$$

Since the spacing of the grid is $1/m \leq \Delta$, there exists a grid point $\widetilde{\theta} \in G$ nearest to θ^* such that

$$|\widetilde{\theta} - \theta^*| \leq \frac{1}{2m} \leq \frac{\Delta}{2}.$$

Now fix any round t.
Case 1: If $y_t = +1$, then

$$y_t = h_{\theta^*}(x_t) = \operatorname{sign}(x_t - \theta^*)$$

and the Δ-separation assumption implies

$$x_t \geq \theta^* + \Delta.$$

Hence

$$x_t - \widetilde{\theta} \geq (\theta^* + \Delta) - \widetilde{\theta} \geq \Delta - |\widetilde{\theta} - \theta^*| \geq \frac{\Delta}{2} > 0.$$

Therefore,

$$h_{\widetilde{\theta}}(x_t) = \operatorname{sign}(x_t - \widetilde{\theta}) = +1 = y_t.$$

Case 2: If $y_t = -1$, then similarly the realizability and separation assumptions imply

$$x_t \leq \theta^* - \Delta.$$

Thus

$$x_t - \widetilde{\theta} \leq (\theta^* - \Delta) - \widetilde{\theta} \leq -\Delta + |\widetilde{\theta} - \theta^*| \leq -\frac{\Delta}{2} < 0,$$

so

$$h_{\widetilde{\theta}}(x_t) = \operatorname{sign}(x_t - \widetilde{\theta}) = -1 = y_t.$$

Since this holds in both cases, we have shown that one hypothesis in

$$H_G = \{h_\theta : \theta \in G\}$$

labels the entire sequence correctly.

2.1.3 Mistake Bound Derivation

Now run the Halving algorithm on the finite class H_G: on each round, predict by the majority vote of the currently surviving hypotheses, and after a mistake eliminate all hypotheses that predicted incorrectly. Because the sequence is realizable by $h_{\widetilde{\theta}} \in H_G$, the Halving theorem implies that the total number of mistakes is at most

$$\log_2 |H_G| = \log_2 |G| = \log_2 \left(\left\lceil \frac{1}{\Delta} \right\rceil + 1 \right).$$

2.1.4 Conclusion

So an explicit online learner is the Halving algorithm over the grid class H_G, with:

- choice of grid:
$$G = \left\{ \frac{k}{m} : k = 0, 1, \ldots, m \right\}, \quad m = \left\lceil \frac{1}{\Delta} \right\rceil;$$
- size of the grid:
$$|G| = \left\lceil \frac{1}{\Delta} \right\rceil + 1;$$
- resulting mistake bound:
$$M \leq \log_2 \left(\left\lceil \frac{1}{\Delta} \right\rceil + 1 \right) = O(\log(1/\Delta)).$$

2.2 Problem 2

2.2.1 Chosen Feature Map

Take $d = 2$ and define
$$\phi(x) = (x, 1) \in \mathbb{R}^2.$$
For the true threshold θ^*, define
$$u^* = \frac{(1, -\theta^*)}{\sqrt{1 + (\theta^*)^2}}.$$
Then u^* is a unit vector, and for every $x \in [0, 1]$,
$$\langle u^*, \phi(x) \rangle = \frac{x - \theta^*}{\sqrt{1 + (\theta^*)^2}}.$$
Therefore,
$$\operatorname{sign}(\langle u^*, \phi(x) \rangle) = \operatorname{sign}(x - \theta^*) = h_{\theta^*}(x).$$
So this representation realizes the threshold function as a linear separator in $\mathbb{R}^2$.

2.2.2 Margin Lower Bound

Now consider any Δ-separated threshold-realizable example (x_t, y_t). Since
$$y_t = \operatorname{sign}(x_t - \theta^*)$$
and
$$|x_t - \theta^*| \geq \Delta,$$
we have
$$y_t(x_t - \theta^*) = |x_t - \theta^*| \geq \Delta.$$
Hence
$$y_t \langle u^*, \phi(x_t) \rangle = \frac{y_t(x_t - \theta^*)}{\sqrt{1 + (\theta^*)^2}} = \frac{|x_t - \theta^*|}{\sqrt{1 + (\theta^*)^2}} \geq \frac{\Delta}{\sqrt{1 + (\theta^*)^2}}.$$
Since $\theta^* \in [0, 1]$, we have
$$1 + (\theta^*)^2 \leq 2,$$
so
$$y_t \langle u^*, \phi(x_t) \rangle \geq \frac{\Delta}{\sqrt{2}}.$$
Thus the sequence is linearly separable with margin
$$\gamma \geq \frac{\Delta}{\sqrt{2}}.$$
In particular, we may take
$$c = \frac{1}{\sqrt{2}}.$$

2.2.3 Norm Bound

For every $x \in [0,1]$,

$$\|\phi(x)\| = \sqrt{x^2+1} \le \sqrt{2}.$$

Therefore we may take

$$R = \sqrt{2}.$$

2.2.4 Mistake Bound Derivation

By the Perceptron theorem, the number of mistakes is at most

$$\frac{R^2}{\gamma^2}.$$

Using

$$R = \sqrt{2} \quad \text{and} \quad \gamma \ge \frac{\Delta}{\sqrt{2}},$$

we obtain

$$M \le \frac{(\sqrt{2})^2}{(\Delta/\sqrt{2})^2} = \frac{2}{\Delta^2/2} = \frac{4}{\Delta^2}.$$

2.2.5 Conclusion

One valid choice is:

- feature map:
$$\phi(x) = (x, 1) \in \mathbb{R}^2;$$
- unit separator:
$$u^* = \frac{(1, -\theta^*)}{\sqrt{1+(\theta^*)^2}};$$
- margin lower bound:
$$\gamma \ge \frac{\Delta}{\sqrt{2}};$$
- norm bound:
$$\|\phi(x)\| \le \sqrt{2};$$
- resulting Perceptron mistake bound:
$$M \le \frac{4}{\Delta^2} = O(1/\Delta^2).$$

2.3 Problem 3

2.3.1 Why This Does Not Contradict Part 1

There is no contradiction between the continuous-threshold impossibility phenomenon from lecture and the result of Part 1 because the two statements concern different families of sequences.
The impossibility phenomenon from lecture applies to arbitrary threshold-realizable sequences. In that setting, examples may appear arbitrarily close to the true threshold θ^*, so the learner may need finer and finer resolution to determine which threshold is correct. In particular, the continuous class $\{h_\theta : \theta \in [0,1]\}$ cannot be treated as a fixed finite class in the worst case.
Part 1 adds the extra assumption of Δ-separation. This changes the problem substantially: every example satisfies

$$|x_t - \theta^*| \ge \Delta.$$

Because of this gap around θ^*, two thresholds that are sufficiently close to each other are indistinguishable on every admissible example. Equivalently, once we know the threshold up to resolution on

the order of Δ, we know all labels in the sequence. That is why a grid of size $O(1/\Delta)$ is enough in Part 1, and the Halving algorithm can be run on that finite effective class.
So the lecture impossibility result rules out a uniform finite-mistake guarantee for the unrestricted continuous threshold class, whereas Part 1 proves a finite-mistake guarantee on the smaller class of Δ-separated realizable sequences.

2.3.2 Comparison of the Two Bounds

The two bounds are

$$M_{\text{Halving}} = O(\log(1/\Delta)) \quad \text{and} \quad M_{\text{Perceptron}} = O(1/\Delta^2).$$

They scale differently because they come from two different kinds of arguments.
Part 1: The Halving argument is combinatorial. After imposing Δ-separation, the continuous threshold class can be replaced by a finite grid of size $|G| = O(1/\Delta)$. The Halving theorem then gives a mistake bound logarithmic in the number of candidate hypotheses:

$$M \leq \log_2 |G| = O(\log(1/\Delta)).$$

This argument measures how many distinguishable threshold locations remain once the margin gap is taken into account.
Part 2: The Perceptron argument is geometric. It does not count hypotheses. Instead, it uses the margin bound

$$\gamma = \Omega(\Delta)$$

and the norm bound

$$R = O(1).$$

The Perceptron theorem then yields

$$M \leq \frac{R^2}{\gamma^2} = O(1/\Delta^2).$$

This argument measures how far the examples are from the separating hyperplane in the chosen linear representation.

2.3.3 Interpretation

The logarithmic bound from Part 1 is sharper for this particular problem because it exploits the special one-dimensional threshold structure very directly. Once the threshold is localized to one of only $O(1/\Delta)$ relevant positions, the problem is essentially finite.
The Perceptron bound is more general but also looser here. It applies through a generic margin analysis for linear separators, and that analysis depends on $1/\gamma^2$. Since the margin is only of order Δ, the resulting guarantee is of order $1/\Delta^2$.

2.3.4 Conclusion

Part 1 and the lecture impossibility result are compatible because Part 1 assumes an additional structural condition, namely Δ-separation. Under that condition, the threshold problem has a finite effective resolution and admits a logarithmic Halving bound. The Perceptron analysis gives a different guarantee because it measures geometric margin rather than the number of distinguishable thresholds, which is why it scales as $O(1/\Delta^2)$ instead of $O(\log(1/\Delta))$.

2.4 Problem 4

2.4.1 Problems in the AI Argument

There are several mathematical problems with the quoted argument.
Problem 1: The class of continuous thresholds does *not* literally have size $O(1/\Delta)$. The hypothesis class

$$\{h_\theta : \theta \in [0, 1]\}$$

is still uncountable. What is true is weaker: for each fixed Δ, one can build a finite grid $G \subseteq [0,1]$ with $|G| = O(1/\Delta)$ such that every Δ-separated threshold-realizable sequence is also realizable by some hypothesis in

$$H_G = \{h_\theta : \theta \in G\}.$$

So the correct statement is about the existence of a finite surrogate class on a restricted family of sequences, not about the cardinality of the original continuous class.

Problem 2: Even if such a finite class H_G exists, it does not follow that *every* online learner enjoys the Halving bound. The Halving theorem gives an $O(\log |H_G|)$ mistake bound for the *Halving algorithm run on* H_G. It says nothing about arbitrary learners.

In fact, the claim "any online learner" is false. Consider the learner that always predicts -1. Fix any $0 < \Delta \leq 1$, and consider the sequence

$$x_t = 1, \qquad y_t = +1 \qquad \text{for } t = 1, \ldots, T.$$

This sequence is Δ-separated threshold-realizable by $\theta^* = 0$, because

$$h_{\theta^*}(x_t) = \text{sign}(1 - 0) = +1 \quad \text{and} \quad |x_t - \theta^*| = 1 \geq \Delta.$$

But the always--1 learner makes T mistakes. Hence no statement of the form "every online learner makes $O(\log(1/\Delta))$ mistakes" can be true.

Problem 3: The phrase "including Perceptron" is therefore unsupported. Perceptron does not inherit the Halving bound just because a finite consistent grid exists. Its guarantee comes from the margin analysis of Problem 2, which yields a bound of order $O(1/\Delta^2)$.

2.4.2 Corrected Statement

A correct statement is the following:

For every $\Delta > 0$, there exists a finite grid $G \subseteq [0,1]$ with $|G| = O(1/\Delta)$ such that the Halving algorithm run on the finite class

$$H_G = \{h_\theta : \theta \in G\}$$

makes at most $O(\log(1/\Delta))$ mistakes on every Δ-separated threshold-realizable sequence. By contrast, the Perceptron guarantee obtained from the margin argument is $O(1/\Delta^2)$.

2.4.3 Proof of the Corrected Statement

Let

$$m = \left\lceil \frac{1}{\Delta} \right\rceil \quad \text{and} \quad G = \left\{ \frac{k}{m} : k = 0, 1, \ldots, m \right\}.$$

Then

$$|G| = m + 1 = \left\lceil \frac{1}{\Delta} \right\rceil + 1 = O(1/\Delta).$$

Now consider any Δ-separated threshold-realizable sequence. By definition, there exists $\theta^* \in [0,1]$ such that

$$y_t = h_{\theta^*}(x_t) \quad \text{and} \quad |x_t - \theta^*| \geq \Delta \qquad \text{for all } t.$$

Choose a grid point $\widetilde{\theta} \in G$ nearest to θ^*. Since the grid spacing is $1/m \leq \Delta$, we have

$$|\widetilde{\theta} - \theta^*| \leq \frac{1}{2m} \leq \frac{\Delta}{2}.$$

We claim that $h_{\widetilde{\theta}}(x_t) = y_t$ for every round t.

Case 1: If $y_t = +1$, then $x_t \geq \theta^* + \Delta$. Therefore

$$x_t - \widetilde{\theta} \geq (\theta^* + \Delta) - \widetilde{\theta} \geq \Delta - |\widetilde{\theta} - \theta^*| \geq \frac{\Delta}{2} > 0.$$

Hence

$$h_{\widetilde{\theta}}(x_t) = +1 = y_t.$$

Case 2: If $y_t = -1$, then $x_t \le \theta^* - \Delta$. Therefore

$$x_t - \widetilde{\theta} \le (\theta^* - \Delta) - \widetilde{\theta} \le -\Delta + |\widetilde{\theta} - \theta^*| \le -\frac{\Delta}{2} < 0.$$

Hence

$$h_{\widetilde{\theta}}(x_t) = -1 = y_t.$$

So the entire sequence is realizable by one hypothesis in the finite class H_G. Applying the Halving theorem to H_G, the number of mistakes made by the Halving algorithm is at most

$$\log_2 |H_G| = \log_2 \left(\left\lceil \frac{1}{\Delta} \right\rceil + 1 \right) = O(\log(1/\Delta)).$$

Finally, this does *not* imply the same bound for Perceptron. The Perceptron analysis from Problem 2 uses the representation $\phi(x) = (x, 1)$, a norm bound $R \le \sqrt{2}$, and a margin lower bound $\gamma \ge \Delta/\sqrt{2}$. Therefore the Perceptron theorem gives

$$M \le \frac{R^2}{\gamma^2} \le \frac{2}{(\Delta/\sqrt{2})^2} = \frac{4}{\Delta^2} = O(1/\Delta^2).$$

This is a different guarantee obtained by a different argument.

2.4.4 Conclusion

The AI argument fails because it confuses an infinite class with a finite surrogate class, and then incorrectly transfers a Halving guarantee to arbitrary learners. The correct logarithmic mistake bound is an existence statement for a specific learner, namely Halving on a carefully chosen grid. Perceptron is governed instead by the margin analysis, which yields the weaker bound $O(1/\Delta^2)$.

3 Part C: Perceptron — Implementation and Experiments

3.1 Question 1: Data Generation

To generate data with a known margin and a known norm bound, I fixed the true separator to

$$u = e_1 = (1, 0, \ldots, 0) \in \mathbb{R}^d$$

and then sampled each example in the form

$$x_t = y_t \gamma u + \rho_t v_t,$$

where:

- $y_t \in \{-1, +1\}$ is sampled uniformly,
- v_t is a unit vector orthogonal to u,
- $0 \le \rho_t \le \sqrt{R^2 - \gamma^2}$.

This construction gives an exact signed margin:

$$y_t \langle u, x_t \rangle = y_t \langle u, y_t \gamma u + \rho_t v_t \rangle = \gamma,$$

because $\langle u, v_t \rangle = 0$ and $\|u\| = 1$. It also enforces the norm bound

$$\|x_t\|^2 = \gamma^2 + \rho_t^2 \le R^2.$$

In all experiments below I set

$$R = 1,$$

so the theoretical mistake bound becomes

$$\frac{R^2}{\gamma^2} = \frac{1}{\gamma^2}.$$

The implementation runs the standard online Perceptron with update rule

$$w_{t+1} = \begin{cases} w_t + y_t x_t, & \text{if } y_t \langle w_t, x_t \rangle \le 0, \\ w_t, & \text{otherwise.} \end{cases}$$

I initialized $w_1 = 0$ and used fixed random seeds for reproducibility.

3.2 Question 2: Scaling with $1/\gamma^2$

For the main scaling experiment, I fixed $d = 25$ and swept γ over

$$\{0.50, 0.35, 0.25, 0.18, 0.14, 0.11, 0.09, 0.07, 0.055\}.$$

For each value of γ, I generated a sequence whose length was a constant multiple of $1/\gamma^2$, ran 48 independent trials, and recorded the number of Perceptron mistakes.
The empirical relationship is shown in Figure 1. A log-log fit of the average mistake count versus $1/\gamma^2$ gave slope

$$0.998,$$

which is essentially linear scaling in $1/\gamma^2$. So the experiments agree very closely with the theorem's prediction about how the mistake count should grow as the margin shrinks.

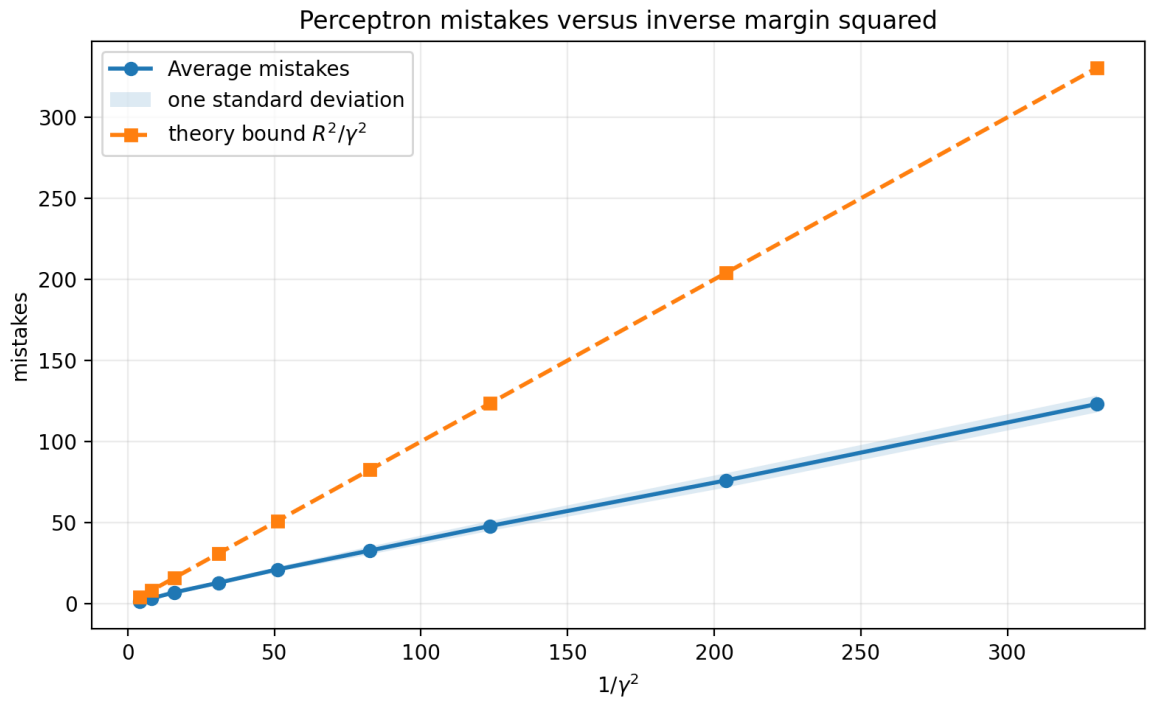

Figure 1: Average Perceptron mistakes versus $1/\gamma^2$, together with the theoretical bound R^2/γ^2.

3.3 Question 3: Tightness of the Bound

The bound has the correct order, but it is not tight in constant for these random sequences. Across the same sweep, the average mistake count was between about

$$0.344 \cdot \frac{R^2}{\gamma^2} \quad \text{and} \quad 0.436 \cdot \frac{R^2}{\gamma^2},$$

with mean ratio

$$0.393.$$

So in practice the Perceptron did noticeably better than the worst-case upper bound, even though the growth rate still tracked $1/\gamma^2$ very closely.
Figure 2 makes this more explicit: the ratio stays well below 1 throughout the sweep, so the theorem is conservative on this data distribution.

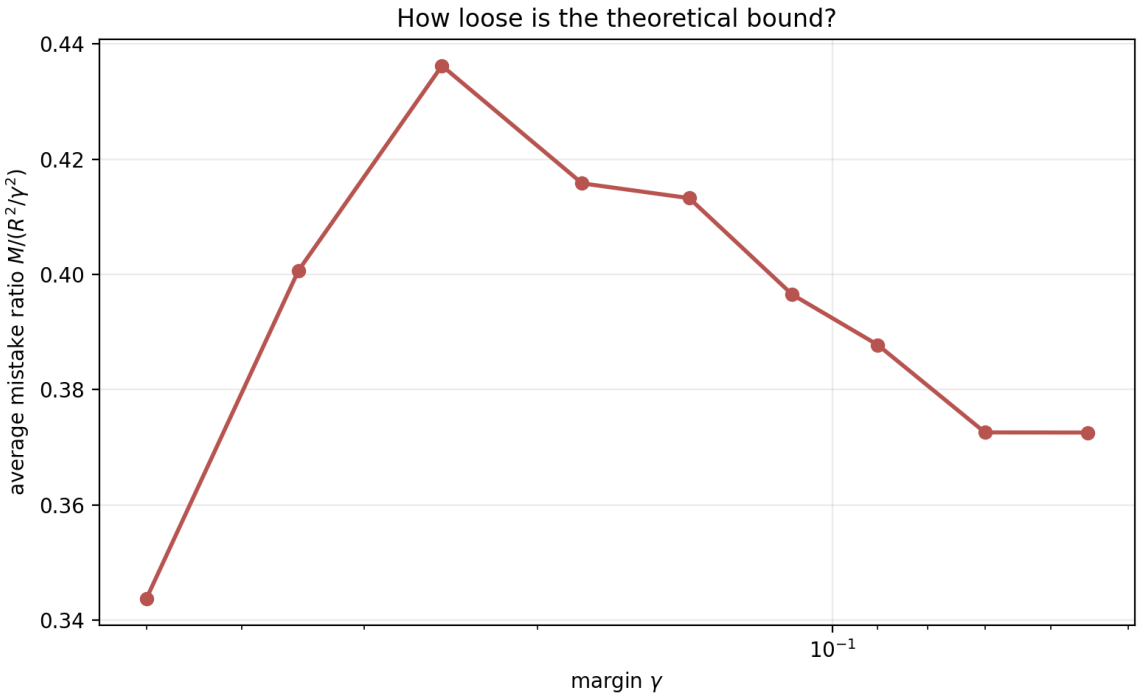

Figure 2: Average mistake count divided by the theoretical bound R^2/γ^2. The observed ratio stays well below 1.

3.4 Question 4: Dependence on the Dimension

To test dimension dependence, I fixed

$$\gamma = 0.08$$

and varied

$$d \in \{2, 5, 10, 20, 50, 100, 200\}.$$

For each dimension I ran 36 trials on a sequence long enough to allow the Perceptron to realize most of its mistakes.

The result is shown in Figure 3. I did *not* observe any explosive dependence on d. The mean mistake count rose from about 14.1 at $d = 2$ to roughly 64–66 for $d = 50, 100, 200$, and the change from $d = 20$ to $d = 200$ was only about 3.3%. So the data are not perfectly flat in d, but once the dimension is moderately large, the mistake count essentially plateaus instead of growing with dimension. That is qualitatively consistent with the theorem being dimension-free.

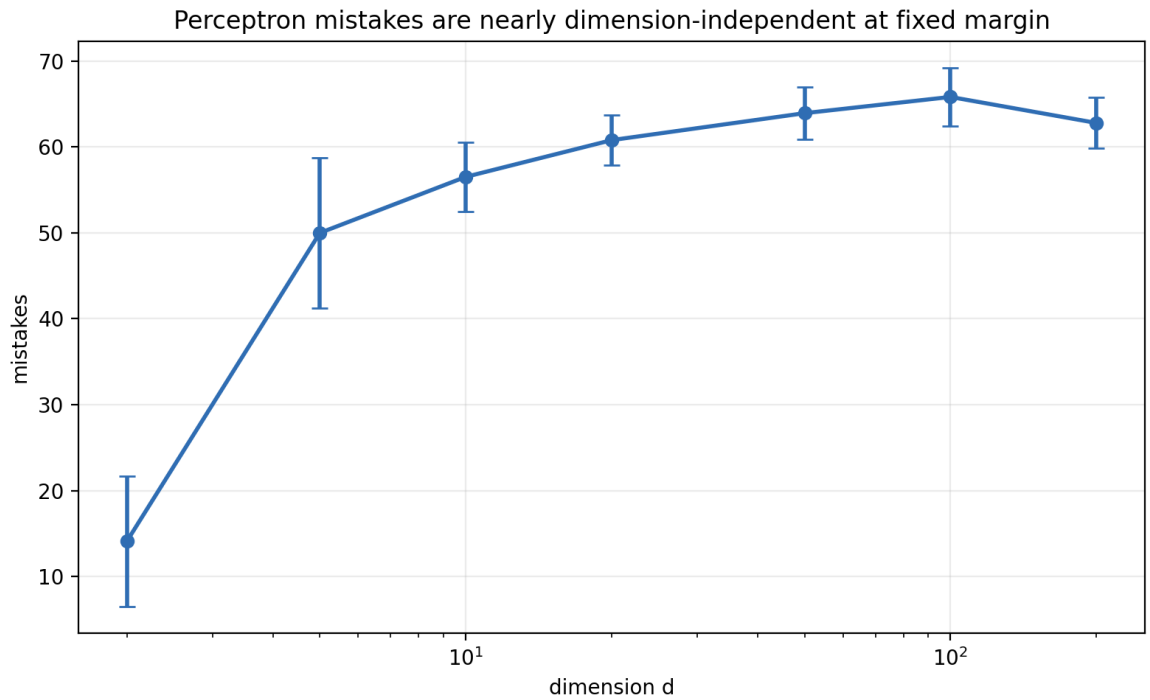

Figure 3: Average mistakes as the ambient dimension changes at fixed margin $\gamma = 0.08$. The curve rises at very small dimensions and then largely flattens.

3.5 Question 5: What Happens as $\gamma \to 0$?

For a smaller-margin sweep, I used

$$\gamma \in \{0.20, 0.15, 0.12, 0.10, 0.08, 0.06, 0.05, 0.04, 0.03\}$$

with $d = 25$. The behavior is shown in Figure 4. As γ decreased from 0.20 to 0.03, the average mistake count increased from about

$$11.46$$

to about

$$388.96.$$

This matches the lecture intuition from the threshold counterexample. When γ is small, the examples lie very close to the decision boundary, so the learner receives only weak geometric separation. The Perceptron still has a finite mistake guarantee for any fixed positive γ, but the guarantee deteriorates rapidly as $\gamma \to 0$. In that sense, the small-margin regime is the finite-margin shadow of the no-margin threshold phenomenon from lecture.

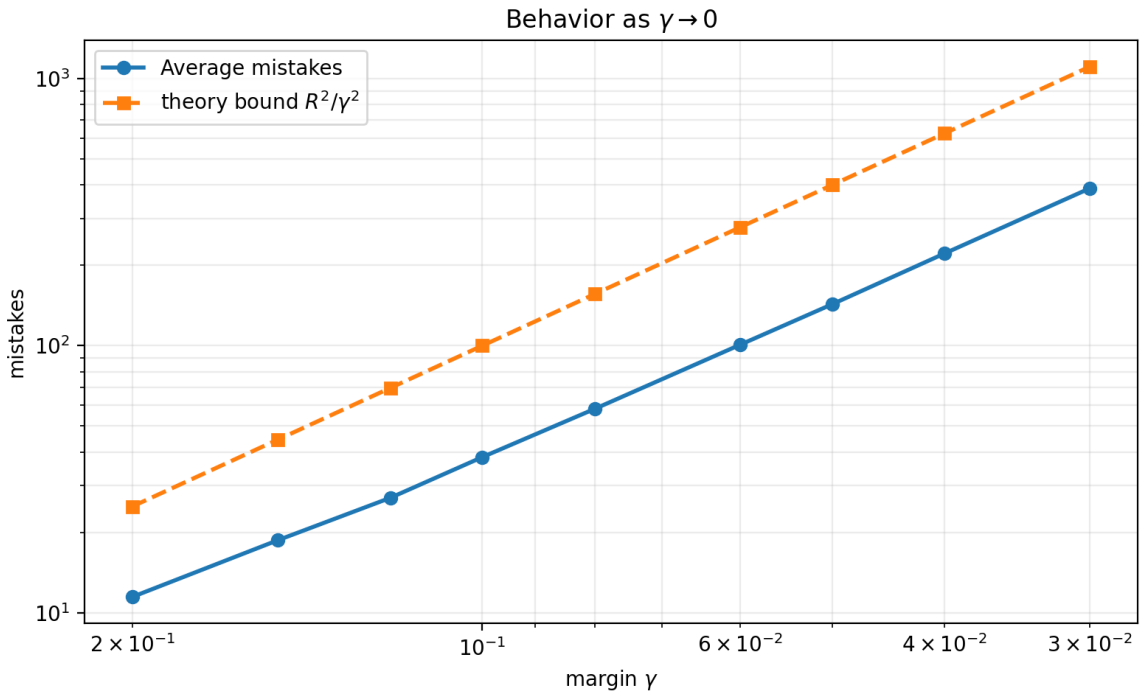

Figure 4: Average mistakes and the theoretical bound as $\gamma \to 0$. Both grow rapidly, reflecting the difficulty of learning from near-boundary examples.

3.6 Question 6: How I Verified the Implementation

If AI helps write code for this part, I do not want to trust the first draft blindly. I built three checks directly into the experiment script:

- **Single-update test:** starting from $w = 0$, one misclassified positive example should produce exactly the update $w \leftarrow x$.
- **Generator invariant test:** every generated sequence is checked to ensure

  $$y_t \langle u, x_t \rangle \geq \gamma$$

  and

  $$\|x_t\| \leq R$$

  up to numerical tolerance.
- **Tiny separable-dataset convergence test:** on a hand-written separable dataset, repeated passes of the Perceptron must end with zero training mistakes.

These checks would catch subtle bugs such as dropping the factor y_t in the update rule, flipping the sign in the margin test, or writing a data generator that silently violates the claimed margin.

3.7 Code

The experiment code for this part is stored at the relative path `hw1/code/perceptron_experiments.py`. The generated numerical summary is stored at `hw1/code/perceptron_summary.json`, and the plots used in this section are stored under `hw1/figures/`.

4 Part D: AI Usage Report

Disclaimer: I did not use AI for this part.

4.1 Q1

Actually I use AI(Codex with GPT 5.4 X-High) for every point you mentioned in the question.

4.2 Q2

There is one interesting case when I set up the skill and ask AI to design the pipeline for writing the proofs.
I use VSCode, and codex is embedded inside. To quickly review the proofs I have to use the vscode pdf extension. At first, The layout is problematic, like no title, duplicated section numbers. I ask AI to check whether it's the tex error or the compiler error, and it found they were all good, and checked the pdf itself by some specific parser. Then it found the problem is the pdf extension, and it suggested me to use another one. After I switched to the new pdf extension, the layout is fixed and looks good.

4.3 Q3

Codex is really good at writting proofs, but bad at styling things. So I write some prompts to make the proof more organized and structured. like subsections and titles. But the proof itself is still not so perfect as human writing, so I will keep update add finding a good skill to make it more clear.

4.4 Q4

This problem set is not so hard so I just roughly checked the results.

4.5 Misc

In this assignment, I only use codex because I'm more famailiar with it. Next time I will investigate Claude Code. And I also spent a lot of time dealing with some connection errors for codex on MacOS, and finally I gave up and put it to my linux server. Hope they could fix the problem soon.

Skills that I built for this assignment and their brief descriptions:

- **/lt-proof** A complete pipeline to write a single proof, with component design and self-checking.
- **/lt-latex-homework-format** A skill that make sure the proof is aligned with required format.
- **/lt-compile-pdf** A command to quickly compile the tex file.

Chain of skill is also interesting...